

文章笔记

文章笔记：Evaluating Deep Agents — Our Learnings

基于 LangChain 官方博客整理

2026 年 4 月 2 日

文章作者/来源：LangChain

发布日期：2025

阅读时长：约 15 分钟

文章链接：<https://blog.langchain.com/evaluating-deep-agents-our-learnings/>

项目主页：<https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：为什么 Deep Agent 的评测如此困难？	2
1.1 术语定义	2
1.2 本章小结	3
2 模式一：定制化测试逻辑	3
2.1 传统评测的局限性	3
2.2 为什么 Deep Agent 需要定制化？	3
2.3 Pytest/Vitest 集成实践	3
2.4 本章小结	4
3 模式二：单步评测 — 高效的“单元测试”	4
3.1 单步评测的核心思想	4
3.2 适用场景	5
3.3 LangGraph 中断机制	5
3.4 本章小结	5
4 模式三：完整轮次评测 — 端到端验证	6
4.1 完整轮次的三个检查维度	6
4.1.1 轨迹检查 (Trajectory)	6
4.1.2 最终响应检查 (Final Response)	6
4.1.3 其他状态检查 (Other State)	6
4.2 LangSmith Trace 可视化	6
4.3 本章小结	7
5 模式四：多轮对话评测 — 模拟真实交互	7
5.1 多轮评测的核心挑战	7
5.2 条件分支策略	7
5.3 多轮评测的设计建议	8
5.4 本章小结	8
6 模式五：环境隔离与可复现性	8
6.1 环境对 Deep Agent 评测的关键影响	8
6.2 编程 Agent 的环境隔离策略	8
6.3 Mock API 请求：更快、更可控	9
6.4 本章小结	9
7 评测策略综合对比	9
8 总结与延伸	10
8.1 核心收获	10
8.2 与软件测试体系的对应关系	10
8.3 拓展阅读	11

1 引言：为什么 Deep Agent 的评测如此困难？

随着 LLM Agent 从简单的“问答式”应用演变为能够长时间自主运行、维护复杂状态的“Deep Agent”，评测方法也必须相应升级。LangChain 团队在过去一个月内发布了四个基于 Deep Agent harness 构建的应用：

- **DeepAgents CLI**: 编程 Agent
- **LangSmith Assist**: LangSmith 平台内的辅助 Agent
- **Personal Email Assistant**: 能从交互中学习用户偏好的邮件助手
- **Agent Builder**: 由 meta deep agent 驱动的低代码 Agent 构建平台

在为这些应用编写评测用例的过程中，LangChain 团队总结出五大核心模式，涵盖了从**单步验证**到**多轮对话模拟**、从**轨迹检查**到**环境隔离**的完整评测体系。

Deep Agent 与传统 LLM 应用的根本区别

传统 LLM 评测假设每个测试用例可以用**相同的评测器**打分；而 Deep Agent 由于具备状态管理、工具调用、多步推理等能力，每个测试用例可能需要**完全不同的断言逻辑 (assertion logic)**。这是本文所有讨论的出发点。

1.1 术语定义

在进入具体评测模式之前，需要先统一本文使用的关键术语。LangChain 团队从两个维度对评测进行分类：**Agent 的运行方式**和**可测试的对象**。

表 1: Agent 运行方式分类

运行方式	说明
Single Step (单步)	限制 Agent 核心循环只执行一步，确定下一个要执行的动作
Full Turn (完整轮次)	让 Agent 在单个输入上完整运行，可能包含多次工具调用迭代
Multiple Turns (多轮)	让 Agent 多次完整运行，模拟用户与 Agent 之间的多轮对话

表 2: 可测试对象分类

测试对象	说明
Trajectory (轨迹)	Agent 调用的工具序列及其参数
Final Response (最终响应)	Agent 返回给用户的最终回复
Other State (其他状态)	Agent 运行过程中产生的文件、artifacts 等副产物

为什么需要区分运行方式和测试对象？

这两个维度是正交的。同一个测试用例可能在 Full Turn 模式下运行，但只检查 Trajectory；也可能在 Single Step 模式下运行，同时检查 Final Response 和 Other State。理解这种正交关系有助于设计出更精准的测试矩阵。

1.2 本章小结

Deep Agent 评测的核心挑战在于**状态性 (statefulness)** 和**长程性 (long-horizon)**。传统的 input-output 评测范式无法覆盖 Agent 的工具调用轨迹、中间状态和副产物。LangChain 提出的五大模式从不同粒度和视角切入，构成了一套相对完整的评测框架。

2 模式一：定制化测试逻辑

2.1 传统评测的局限性

传统 LLM 评测遵循一个简洁的三步流程：

1. 构建测试数据集
2. 编写评测器 (evaluator)
3. 将应用在数据集上运行，用评测器对输出打分

在这个范式下，所有数据点被“一视同仁”——通过相同的应用逻辑处理，由相同的评测器打分。这对于分类、摘要、翻译等任务是足够的，但对 Deep Agent 来说远远不够。

2.2 为什么 Deep Agent 需要定制化？

Deep Agent 的“成功标准”因任务而异，而且往往涉及对 Agent 轨迹和状态的具体断言。文章给出了一个生动的例子：

假设有一个日历调度 Deep Agent，用户说“记住永远不要在早上 9 点之前安排会议”。我们需要验证的不仅仅是 Agent 的回复，还有：

1. Agent 是否调用了 `edit_file` 工具修改了 `memories.md` 文件？
2. Agent 是否在最终回复中确认了偏好已记录？
3. `memories.md` 文件中是否确实包含了关于“不安排 9 点前会议”的信息？

常见误区：只检查最终回复

很多开发者在评测 Agent 时只关注最终输出的文本质量，忽略了 Agent 是否真正完成了任务。一个 Agent 可能回复“好的，已经帮你记住了”但实际上并没有调用任何工具来更新记忆文件。**对 Deep Agent 而言，回复内容和实际行为必须同时验证。**

2.3 Pytest/Vitest 集成实践

LangSmith 提供了 Pytest 和 Vitest 集成，允许开发者为每个测试用例编写不同的断言逻辑。下面是文章给出的核心代码示例：

```
1 @pytest.mark.langsmith
2 def test_remember_no_early_meetings() -> None:
3     user_input = "I don't want any meetings scheduled before 9 AM ET"
4     t.log_inputs({"question": user_input})
5     response = run_agent(user_input)
```

```

6     t.log_outputs({"outputs": response})
7     agent_tool_calls = get_agent_tool_calls(response)
8     # 验证 Agent 调用了 edit_file 工具更新 memories.md
9     assert any(
10         [tc["name"] == "edit_file"
11          and tc["args"]["path"] == "memories.md"
12          for tc in agent_tool_calls]
13     )
14     # LLM-as-judge 验证最终回复是否确认了偏好记录
15     communicated_to_user = llm_as_judge_A(response)
16     t.log_feedback(key="communicated_to_user",
17                   score=communicated_to_user)
18     # LLM-as-judge 验证记忆文件是否包含正确信息
19     memory_updated = llm_as_judge_B(response)
20     t.log_feedback(key="memory_updated",
21                   score=memory_updated)

```

Listing 1: LangSmith Pytest 集成 — 定制化 Agent 测试

代码中有几个关键设计值得注意：

- `t.log_inputs` / `t.log_outputs`：将输入输出记录到 LangSmith，便于后续调试和追踪
- `get_agent_tool_calls`：提取 Agent 的工具调用轨迹，用于轨迹断言
- `llm_as_judge`：对于难以用规则匹配的检查项（如“回复是否确认了偏好”），使用 LLM 作为裁判来打分
- `t.log_feedback`：将评分结果记录为结构化反馈，而非简单的 pass/fail

定制化测试的三层验证模型

对 Deep Agent 的每个测试用例，建议从三层进行验证：

1. **行为层**：Agent 是否调用了正确的工具、传入了正确的参数？（轨迹断言）
2. **沟通层**：Agent 是否在回复中准确传达了执行结果？（LLM-as-judge）
3. **效果层**：Agent 的操作是否真正产生了预期的副作用？（状态检查）

2.4 本章小结

定制化测试逻辑是 Deep Agent 评测的基石。传统的“统一评测器”模式无法覆盖 Agent 的多维度行为。通过 Pytest/Vitest 集成，开发者可以为每个测试用例编写独立的断言逻辑，结合轨迹检查、LLM-as-judge 和状态验证，实现全面的质量保障。

3 模式二：单步评测 — 高效的“单元测试”

3.1 单步评测的核心思想

LangChain 团队报告，在他们的 Deep Agent 评测用例中，约一半采用单步评测模式。单步评测的核心问题是：

给定特定的输入消息序列，LLM 做出的下一步决策是否正确？

这类似于软件工程中的**单元测试**——不关心整个系统的端到端行为，只验证单个决策点的正确性。

3.2 适用场景

单步评测特别适合验证以下类型的决策：

- Agent 是否选择了正确的工具来搜索会议时间？
- Agent 是否检查了正确的目录内容？
- Agent 是否更新了其记忆？
- Agent 是否正确地拒绝了不合理的请求？

为什么回归问题多发生在单个决策点？

LangChain 团队的经验表明，Agent 的回归（regression）更多发生在单个决策点而非完整执行序列上。这与软件工程中 bug 倾向于出现在“接口边界”的规律一致。当 prompt 或模型版本更新时，Agent 在特定场景下的工具选择和参数生成最容易出现偏差。

3.3 LangGraph 中断机制

如果使用 LangGraph 框架，可以利用其 streaming 能力在 Agent 执行第一个工具调用后立即中断，从而高效地实现单步评测：

```
1 @pytest.mark.langsmith
2 def test_single_step() -> None:
3     state_before_tool_execution = await agent.ainvoke(
4         inputs,
5         # interrupt_before: 指定在哪些节点前中断
6         # 在 tools 节点前中断可以检查工具调用参数
7         interrupt_before=["tools"]
8     )
9     # 获取 Agent 的消息历史，包括最新的工具调用
10    print(state_before_tool_execution["messages"])
```

Listing 2: LangGraph 单步评测 — interrupt_before 机制

单步评测的效率优势

单步评测不仅执行速度快（通常只需一次 LLM 调用），还能大幅节省 token 消耗。对于需要频繁运行的回归测试套件来说，这意味着显著更低的成本和更快的反馈循环。LangChain 建议将单步评测作为 CI/CD pipeline 中的第一道关卡。

3.4 本章小结

单步评测是 Deep Agent 评测中最高效的手段。它聚焦于单个决策点的正确性，类似于单元测试。通过 LangGraph 的 interrupt_before 机制可以优雅地实现。建议将约一半的测试用例设计为单步评测，覆盖 Agent 的关键决策分支。

4 模式三：完整轮次评测 — 端到端验证

4.1 完整轮次的三个检查维度

完整轮次评测让 Agent 从接收输入到返回最终结果完整运行一次。与单步评测的“单元测试”类比不同，完整轮次更像是**集成测试**，关注 Agent 多步协作后的整体效果。LangChain 总结了三个检查维度：

4.1.1 轨迹检查 (Trajectory)

验证 Agent 在整个执行过程中是否调用了必要的工具，**但不要求严格的调用顺序**。例如日历调度 Agent 可能需要多次工具调用才能找到所有参与者都空闲的时间段——我们只关心它最终是否查询了所有相关日历，而不关心查询的顺序。

4.1.2 最终响应检查 (Final Response)

对于编程和研究等开放性任务，Agent 到达目标的**路径**可能有很多种，此时最终输出的质量比具体路径更重要。这与传统的“标准答案”评测模式有根本区别。

4.1.3 其他状态检查 (Other State)

很多 Deep Agent 不仅仅产生文本回复，还会创建 artifacts（文件、代码、配置等）。这些副产物也需要纳入评测范围：

表 3: 不同类型 Agent 的状态检查策略

Agent 类型	状态检查方式
编程 Agent	读取并测试 Agent 生成的代码文件（运行单元测试、检查语法等）
研究 Agent	验证 Agent 是否找到了正确的链接或信息来源
邮件 Agent	检查 Agent 是否正确更新了用户偏好文件
调度 Agent	验证日历中是否实际创建了正确的事件

轨迹检查不要过度约束

常见错误是对工具调用的**顺序**做严格断言。Agent 可能因为模型版本更新、temperature 差异等原因改变工具调用顺序，但只要最终结果正确，这种顺序变化是可以接受的。过度约束会导致**脆弱测试 (flaky tests)**，增加维护成本。建议使用 `any()` 而非 `assertEqual(trajecory, expected_trajecory)` 来做轨迹断言。

4.2 LangSmith Trace 可视化

LangSmith 提供了完整轮次的 trace 可视化能力，可以查看：

- 高层指标：延迟 (latency) 和 token 消耗
- 细粒度步骤：每次模型调用和工具调用的详细信息

- 失败定位：当测试失败时，trace 视图可以快速定位到具体出错的步骤

这种从宏观到微观的多层级追踪对于调试复杂 Agent 行为至关重要。

4.3 本章小结

完整轮次评测提供了 Agent 行为的全景视图。通过轨迹、最终响应和其他状态三个维度的检查，可以全面评估 Agent 的端到端表现。关键原则是：**检查结果而非路径**，对轨迹做适度宽松的断言以避免脆弱测试。

5 模式四：多轮对话评测 — 模拟真实交互

5.1 多轮评测的核心挑战

真实场景中，用户往往与 Agent 进行多轮交互。例如先问“帮我安排明天的会议”，然后根据 Agent 的回复进一步说“改到下午 3 点”。然而，多轮评测面临一个根本性问题：

硬编码多轮输入的脆弱性

如果天真地将多轮对话的用户输入硬编码为一个固定序列，当 Agent 在第一轮的回复偏离预期时，后续的硬编码输入将与上下文脱节，导致整个测试变得无意义。例如，如果 Agent 没有按预期询问“你想安排什么时间？”，而是直接建议了一个时间，那么硬编码的“下午 3 点”回复就毫无意义了。

5.2 条件分支策略

LangChain 团队采用的解决方案是在 Pytest/Vitest 测试中引入**条件逻辑**：

1. 运行第一轮，检查 Agent 输出
2. 如果输出符合预期，继续运行下一轮
3. 如果输出不符合预期，**提前终止测试** (early fail)

这种方式的优势在于：

- 不需要穷举 Agent 的所有可能分支
- 每一轮都有独立的检查点 (checkpoint)
- 失败时可以精确定位到哪一轮出了问题

隔离测试特定轮次

如果需要单独测试第二轮或第三轮的行为，可以直接用适当的初始状态 (initial state) 启动测试，跳过前面的轮次。这在 LangGraph 中特别方便，因为 Agent 的状态可以被序列化和恢复。这种技巧让多轮测试的编写和维护成本大幅降低。

5.3 多轮评测的设计建议

根据 LangChain 的经验，多轮评测的设计应遵循以下原则：

表 4: 多轮评测设计原则

原则	说明
最少轮次	每个测试用例只包含验证目标行为所需的最少轮数
条件推进	每一轮结束后验证输出，只在预期路径上推进
早期失败	不符合预期时立即终止，避免浪费资源在无效状态上运行
状态快照	支持从任意中间状态恢复，便于隔离测试特定轮次

5.4 本章小结

多轮对话评测模拟了真实的用户交互场景，但面临输入与上下文脱节的挑战。通过条件分支策略和状态快照机制，可以在保持测试有效性的同时控制复杂度。关键是**不要试图穷举所有对话路径**，而是选择最关键的场景进行有针对性的测试。

6 模式五：环境隔离与可复现性

6.1 环境对 Deep Agent 评测的关键影响

Deep Agent 具有状态性，能够读写文件、调用 API、修改系统配置。这意味着如果测试环境没有正确隔离，前一个测试用例的副作用会污染后续测试，导致难以复现的测试失败（flaky tests）。

Deep Agent 评测的黄金法则

每个 eval 运行必须获得一个**全新、干净的环境**，以确保结果可复现。这是 Deep Agent 评测与传统 LLM 评测最大的基础设施差异。

6.2 编程 Agent 的环境隔离策略

文章以编程 Agent 为例介绍了两种隔离策略：

表 5: 环境隔离策略对比

特性	Docker 容器/Sandbox	临时目录
隔离程度	完全隔离（进程、文件系统、网络）	文件系统级隔离
启动开销	较高（秒级）	极低（毫秒级）
适用场景	需要网络隔离或系统级操作的 Agent	仅需文件系统操作的 Agent
代表实现	Harbor / TerminalBench	DeepAgents CLI 临时目录方案

Harbor 与 TerminalBench

Harbor 是一个为编程 Agent 评测设计的平台，TerminalBench 是其提供的评测基准之一。它为每个评测任务启动独立的 Docker 容器，确保 Agent 的文件操作、进程管理和网络访问互不干扰。对于生产级 Agent 评测系统，这类重量级隔离方案是推荐做法。

6.3 Mock API 请求：更快、更可控

除了文件系统隔离，API 调用也需要考虑。LangSmith Assist 需要连接真实的 LangSmith API，但直接调用生产 API 存在两个问题：

1. **速度慢**：网络请求增加延迟，拖慢整个评测流程
2. **成本高**：大量 API 调用可能产生费用
3. **不稳定**：外部服务的状态变化会影响测试结果的可复现性

解决方案是**录制-回放 (record-replay) 模式**：

表 6: API Mock 工具推荐

语言	工具	原理
Python	VCR.py	首次运行录制 HTTP 交互到文件，后续回放
JavaScript	Hono proxy	通过代理应用拦截并回放 fetch 请求

Mock 的隐患：API 变更导致过期

录制-回放模式的一个风险是当上游 API 发生变更时，录制的响应可能已经过时 (stale)。建议定期用真实 API 重新录制一次基准数据，并在 CI 中设置周期性的“真实 API”测试作为兜底。

6.4 本章小结

环境隔离和可复现性是 Deep Agent 评测的基础设施层保障。文件系统操作需要临时目录或容器隔离，API 调用需要录制-回放机制。两者结合可以显著提高评测的速度、成本效率和结果稳定性。

7 评测策略综合对比

综合前面五种模式，下表从多个维度进行对比，帮助读者根据具体场景选择合适的评测策略：

表 7: 五大评测模式综合对比

评测模式	执行成本	覆盖范围	调试难度	适用阶段
定制化测试逻辑	中等	视具体用例	低	全流程
单步评测	低	单决策点	极低	CI/CD 快速验证
完整轮次评测	高	端到端	中等	发布前回归
多轮对话评测	很高	用户交互	较高	关键场景覆盖
环境隔离	基础设施	横切关注点	—	全流程基础

选择评测模式的决策框架

1. 首先确保环境隔离（模式五），这是所有其他模式的前提
2. 主力使用单步评测（模式二）覆盖关键决策点，约占用例的 50%
3. 补充用完整轮次评测（模式三）做端到端验证
4. 精选几个关键场景用多轮评测（模式四）模拟真实交互
5. 贯穿使用定制化测试逻辑（模式一）为每个用例编写针对性断言

8 总结与延伸

8.1 核心收获

LangChain 这篇文章的核心论点可以归纳为一句话：Deep Agent 的评测不能套用传统 LLM 评测的范式，需要从根本上重新设计。具体来说：

1. 测试粒度：从“一个评测器适用所有”到“每个用例定制断言”
2. 运行模式：从“全量运行”到“单步 + 全轮 + 多轮”三级体系
3. 检查维度：从“只看输出”到“轨迹 + 响应 + 状态”三维验证
4. 基础设施：从“无状态评测”到“环境隔离 + API Mock”

8.2 与软件测试测试体系的对应关系

Deep Agent 评测体系与经典的软件测试金字塔有着惊人的对应关系：

表 8: Deep Agent 评测与软件测试金字塔的映射

软件测试层级	Deep Agent 评测模式	建议占比
单元测试	单步评测 (Single Step)	~50%
集成测试	完整轮次评测 (Full Turn)	~30%
端到端测试	多轮对话评测 (Multiple Turns)	~20%

8.3 拓展阅读

- LangSmith 官方文档: docs.smith.langchain.com — 详细的评测集成指南
- LangGraph 文档: langchain-ai.github.io/langgraph — 了解 Agent 状态管理和中断机制
- Harbor / TerminalBench: 编程 Agent 专用评测环境
- VCR.py: vcrpy.readthedocs.io — Python HTTP 录制回放库
- LangChain 博客原文: Evaluating Deep Agents: Our Learnings